

UNIVERSIDADE FEDERAL DO TOCANTINS
CAMPUS PALMAS
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA DE SISTEMAS OPERACIONAIS

COMPARAÇÃO PRÁTICA ENTRE PROCESSOS E THREADS:
um estudo experimental com Python sobre desempenho em tarefas CPU-intensivas

Heitor Fernandes; Matheus Dreher; João Pedro; Maria Eduarda
Universidade Federal do Tocantins – Campus Palmas

RESUMO

Este trabalho apresenta um estudo experimental comparativo entre as abstrações de processos e threads na linguagem Python, sob a perspectiva do sistema operacional Windows 11. O experimento utiliza o cálculo de números primos em um intervalo de 0 a 500.000 como carga de trabalho CPU-intensiva, executada com 1, 2 e 4 unidades de trabalho paralelas (*workers*), com três repetições por cenário para obtenção de médias confiáveis. Os resultados demonstram empiricamente o impacto do GIL (*Global Interpreter Lock*) do CPython sobre o paralelismo com threads, evidenciando que para tarefas de processamento intensivo, a abordagem baseada em processos via biblioteca *multiprocessing* é a única capaz de obter ganho real de desempenho em sistemas multicore. O trabalho discute também as implicações teóricas desses resultados no contexto de escalonamento, troca de contexto e gerência de recursos do sistema operacional.

Palavras-chave: *processos; threads; GIL; Python; sistemas operacionais; concorrência; paralelismo.*

1 INTRODUÇÃO

A capacidade de executar múltiplas tarefas simultaneamente é um dos pilares dos sistemas operacionais modernos. Para isso, o SO disponibiliza duas opções fundamentais ao programador: o processo e a thread. Embora ambos sirvam ao propósito de concorrência, elas são bem diferentes em sua estrutura interna, no custo de criação, no modelo de comunicação e no comportamento frente ao escalonador do sistema operacional.

A escolha entre processos e threads é uma decisão de projeto que impacta o desempenho, a confiabilidade e a complexidade de um software. Em linguagens como C ou Java, essa escolha é orientada principalmente pelo nível de isolamento desejado e pelo overhead aceitável. Em Python, entretanto, existe um fator adicional e

determinante: o GIL (Global Interpreter Lock), um mecanismo interno do interpretador CPython que restringe o paralelismo real entre threads no contexto de tarefas CPU-intensivas.

Este trabalho tem como objetivo investigar, de forma experimental e empírica, as diferenças de desempenho entre processos e threads em Python na execução de uma tarefa de processamento intensivo de CPU — o cálculo de números primos em um intervalo de 0 a 500.000. O experimento varia a quantidade de workers (1, 2 e 4) e mede o tempo de execução em cada configuração, repetindo cada cenário três vezes para obtenção de médias estatisticamente mais confiáveis.

Os resultados são analisados com base em conceitos teóricos de sistemas operacionais, com ênfase nos mecanismos de escalonamento, troca de contexto, espaço de endereçamento e comunicação entre processos. Espera-se que este trabalho contribua para a compreensão prática de como decisões de concorrência afetam o comportamento real das aplicações sob o mandado do sistema operacional.

2 REFERENCIAL TEÓRICO

2.1 Processos

Um processo é a unidade fundamental de execução em um sistema operacional. Tanenbaum e Bos (2015, p. 85) definem processo como "um programa em execução, incluindo os valores atuais do contador de programa, dos registradores e das variáveis". Do ponto de vista do sistema operacional, cada processo possui um espaço de endereçamento exclusivo, composto por regiões de código (text), dados estáticos (data), heap e pilha (stack).

O sistema operacional mantém, para cada processo, uma estrutura de dados denominada PCB (Process Control Block), que armazena o identificador do processo (PID), o estado de execução, os valores dos registradores, as informações de memória virtual, os descritores de arquivos abertos e os dados de contabilização de recursos. Essa estrutura é essencial para que o escalonador possa realizar a troca de contexto entre processos (SILBERSCHATZ; GALVIN; GAGNE, 2018).

A criação de um processo é uma operação custosa, pois exige a alocação de um novo espaço de endereçamento, a inicialização do PCB e a duplicação das estruturas do processo pai. Em sistemas UNIX, esse mecanismo é realizado pela chamada de sistema `fork()`, que cria uma cópia do processo pai. No Windows, o mecanismo equivalente é o `CreateProcess()`, que inicializa um novo processo do zero. Já em Python, a biblioteca `multiprocessing` permite que os processos sejam executados paralelamente contornando a problemática causada pelo GIL.

2.2 Threads

Uma thread, é uma unidade de execução que existe dentro de um processo. Enquanto processos distintos possuem espaços de endereçamento independentes e não compartilham informações, múltiplas threads de um mesmo processo compartilham o mesmo espaço de memória, os mesmos arquivos abertos e os mesmos recursos do

processo pai. Cada thread possui individualmente apenas seu próprio contador de programa, conjunto de registradores e pilha de execução.

Silberschatz, Galvin e Gagne (2018, p. 160) destacam que "a motivação para o uso de threads reside na possibilidade de se obter paralelismo dentro de um único processo, com menor overhead de criação e comunicação em comparação com múltiplos processos". Realmente, a criação de uma thread é significativamente mais rápida do que a de um processo, e a comunicação entre threads se dá diretamente por variáveis compartilhadas, sem necessidade de mecanismos de IPC (Inter-Process Communication).

A contrapartida do compartilhamento de memória é a necessidade de sincronização. O acesso simultâneo de múltiplas threads a uma variável compartilhada sem controle adequado pode resultar em condições de corrida (race conditions), onde duas ou mais threads começam a competir entre si para executar ou modificar um mesmo dado. As race conditions acabam produzindo resultados inconsistentes e não determinísticos, causando travamentos e outros erros. Mecanismos como mutexes, semáforos e monitores são utilizados para garantir exclusão mútua nas regiões críticas do código (STALLINGS, 2018).

2.3 Escalonamento e Troca de Contexto

O escalonador (scheduler) do sistema operacional é o componente responsável por decidir qual processo ou thread ocupa a CPU em cada instante. Em sistemas de tempo compartilhado, o escalonador preemptivo interrompe por um tempo a tarefa em execução e cede a CPU a outra, criando a ilusão de paralelismo mesmo em sistemas com um único núcleo.

A troca de contexto (context switch) é a operação de salvar o estado completo da tarefa que está sendo retirada da CPU, (informações e arquivos como: registradores, ponteiro de pilha e contador de programa) e restaurar o estado da próxima tarefa a ser executada. Stallings (2018) observa que a troca de contexto entre threads do mesmo processo é mais rápida do que entre processos distintos, pois não exige a troca do espaço de endereçamento virtual nem a invalidação do TLB (Translation Lookaside Buffer).

2.4 O GIL do CPython

O CPython, implementação padrão da linguagem Python, incorpora um mecanismo interno denominado GIL (Global Interpreter Lock). O GIL é um mutex que garante que apenas uma thread execute bytecode Python por vez, independentemente do número de núcleos disponíveis no processador.

A existência do GIL está relacionada ao gerenciamento de memória do CPython, que utiliza contagem de referências para controle do ciclo de vida dos objetos. Sem o GIL, operações concorrentes de incremento e decremento do contador de referências por múltiplas threads poderiam corromper as estruturas internas do interpretador. O GIL simplifica essa gestão ao garantir exclusão mútua implícita em nível de interpretador.

A consequência prática do GIL é que threads Python não ganham desempenho em tarefas CPU-intensivas, mesmo em hardware multicore. Beazley (2010) demonstra que em alguns cenários, o uso de múltiplas threads para processamento de CPU pode resultar em desempenho inferior ao da execução sequencial, devido ao overhead de contenção pelo GIL. A biblioteca multiprocessing contorna essa limitação ao criar processos independentes — cada um com seu próprio interpretador e, portanto, seu próprio GIL — permitindo paralelismo real em múltiplos núcleos.

O GIL é frequentemente citado como a principal limitação do Python para aplicações de alto desempenho em processamento paralelo. Para tarefas I/O-bound, o GIL é liberado durante operações de entrada e saída, permitindo que outras threads executem. Para tarefas CPU-bound, o GIL permanece adquirido durante a execução do bytecode, impedindo o paralelismo real entre threads.

(BEAZLEY, 2010, tradução nossa)

3 METODOLOGIA

3.1 Ambiente de Execução

Os experimentos foram realizados em um computador pessoal com sistema operacional Windows 11, utilizando Python 3.14 como ambiente de execução. As bibliotecas empregadas, multiprocessing, threading, time e csv. Essas bibliotecas são todas partes da biblioteca padrão do Python, sem dependências externas, tendo assim reprodutibilidade do experimento em qualquer instalação padrão da linguagem.

Componente	Especificação
Sistema Operacional	Windows 11
Linguagem	Python 3.14
Bibliotecas	multiprocessing, threading, time, csv
Tarefa computacional	Contagem de primos (0 a 500.000)
Configurações testadas	1, 2 e 4 workers
Repetições por cenário	10

Tabela 1 – Ambiente e configuração do experimento

Fonte: Os autores (2025)

3.2 Tarefa Computacional

A tarefa escolhida para o experimento foi a contagem de números primos no intervalo de 0 a 500.000. Essa escolha foi motivada por três critérios: (i) a tarefa é limitada pela CPU-bound, exigindo processamento contínuo sem operações de I/O; (ii) o intervalo é facilmente partido em blocos independentes, adequados para distribuição entre workers; e (iii) o resultado é verificável, qualquer configuração correta deve encontrar exatamente 41.538 números primos no intervalo, o que permite validar a correção do código.

O algoritmo de verificação de primalidade implementado utiliza a otimização da raiz quadrada: para verificar se n é primo, testa-se divisibilidade apenas por divisores ímpares até $\sqrt{n}$. Isso acaba reduzindo a complexidade de $O(n)$ para $O(\sqrt{n})$ por número, tornando o teste mais eficiente sem alterar a natureza CPU-intensiva da carga total.

3.3 Estrutura do Experimento

O experimento compara duas abordagens de concorrência, processos via `multiprocessing.Process` e threads via `threading.Thread`, em três configurações de paralelismo: 1, 2 e 4 workers. Em cada configuração, o intervalo $[0, 500.000]$ é dividido em blocos de tamanho aproximadamente igual, com o último bloco absorvendo o restante da divisão inteira para garantir cobertura completa do intervalo.

Cada combinação de abordagem e número de workers foi executada 10 vezes consecutivas, e o tempo médio das 10 execuções foi registrado como resultado do cenário. O tempo foi medido com `time.perf_counter()`, o relógio de maior resolução disponível no Python, capturando o intervalo entre o lançamento do primeiro worker e a conclusão do último. Os resultados de cada execução foram armazenados em arquivo CSV para análise posterior.

Um aspecto relevante da implementação diz respeito à restrição do Windows sobre o modelo de criação de processos. No Linux, o método padrão é `fork`, que cria uma cópia do processo pai na memória. No Windows, o método é `spawn`, que inicializa um novo interpretador Python do zero e serializa (via `pickle`) as funções e argumentos a serem executados. Por essa razão, as funções worker foram definidas no escopo global do módulo, closures (funções locais) não são colocadas em série pelo `pickle` e causariam erro de execução no Windows.

3.4 Comunicação entre Workers

A forma como cada worker retorna seu resultado difere entre as duas abordagens, mostrando diferenças nas arquiteturas fundamentais entre processos e threads. No caso dos processos, o resultado é retornado através de uma fila de comunicação (`multiprocessing.Queue`), pois processos possuem espaços de endereçamento independentes e não podem compartilhar variáveis diretamente. No caso das threads, o resultado é escrito diretamente em uma posição de uma lista compartilhada, aproveitando o fato de que todas as threads de um mesmo processo compartilham o mesmo espaço de memória.

4 RESULTADOS

4.1 Dados Coletados

A Tabela 2 apresenta os tempos de execução obtidos em cada uma das três repetições por cenário, bem como a média aritmética correspondente. Em todos os cenários testados, o total de primos encontrados foi de 41.538, confirmando a correção da implementação.

Abordagem	Workers	Rep 1 (s)	Rep 2 (s)	Rep 3 (s)	Rep 4 (s)	Rep 5 (s)	Rep 6 (s)	Rep 7 (s)	Rep 8 (s)	Rep 9 (s)	Rep 10 (s)	Média (s)
Processos	1	0.426	0.416	0.428	0.432	0.426	0.423	0.418	0.423	0.418	0.422	0.423
Threads	1	0.360	0.380	0.366	0.420	0.371	0.403	0.443	0.373	0.386	0.380	0.388
Processos	2	0.301	0.299	0.307	0.279	0.291	0.287	0.302	0.303	0.296	0.296	0.296
Threads	2	0.428	0.359	0.362	0.431	0.405	0.401	0.400	0.363	0.381	0.373	0.390
Processos	4	0.191	0.192	0.204	0.232	0.246	0.221	0.208	0.195	0.206	0.196	0.209
Threads	4	0.371	0.378	0.360	0.359	0.355	0.358	0.365	0.363	0.349	0.351	0.361

Tabela 2 – Tempos de execução por abordagem, número de workers e repetição

Fonte: Os autores (2025)

4.2 Análise por Abordagem

Para a abordagem baseada em threads, os tempos médios para 1, 2 e 4 workers foram, respectivamente, 0,388 s, 0,390 s e 0,361 s. A variação entre o melhor dos cenários (mais rápido) e o pior dos cenários (mais lento), é de aproximadamente 8,03%. Esse comportamento indica que não houve aumento de desempenho com o aumento do número de threads, resultado diretamente esperado pelo comportamento do GIL, que impede a execução simultânea de bytecode Python por múltiplas threads em tarefas CPU-bound.

Para a abordagem baseada em processos, os tempos médios para 1, 2 e 4 workers foram, respectivamente, 0,423 s, 0,296 s e 0,209 s. Curiosamente, diferente do comportamento das Threads os processos tiveram um grande ganho de desempenho quanto maior a quantidade de processos. O cenário com 2 processos apresentou uma redução de tempo em aproximadamente 30,01%, já com 4 processos o aumento foi de 50,59% em comparação com o processo de worker único. Esse resultado está associado às características do ambiente de execução — discutidas na Seção 5 — e não contradiz a teoria do paralelismo por processos.

A Tabela 3 compara os tempos médios entre as duas abordagens para cada configuração de workers, destacando a diferença relativa.

Workers	Processos (s)	Threads (s)	Diferença (%)
1	0,423	0,388	+9,02% (processos mais lentos)
2	0,296	0,390	-24,10% (processos mais rápidos)

Workers	Processos (s)	Threads (s)	Diferença (%)
4	0,209	0,361	-42,11% (processos mais rápidos)

Tabela 3 – Comparação de tempo médio entre processos e threads

Fonte: Os autores (2025)

5 DISCUSSÃO

5.1 Interpretação dos Resultados

O resultado mais relevante deste experimento está no contraste claro entre o paralelismo real (processos) e a concorrência simulada (threads), se assemelhando a duas grandezas inversamente proporcionais.

No caso dos processos, os dados demonstraram um grande ganho de desempenho, com 1 processo, o tempo foi o maior registrado na abordagem (0,423 s) devido ao custo de inicialização da infraestrutura do módulo multiprocessing. Contudo, à medida que mais workers foram adicionados (2 e 4), a carga de cálculo dos números primos foi distribuída de forma efetiva entre os núcleos físicos do processador. O tempo caiu para 0,296 s e, finalmente, para 0,209 s. Isso prova que o *overhead* de criação de processos no Windows (modelo *spawn*) foi amplamente compensado pelo ganho de computação paralela pura.

Por outro lado, no cenário das threads foi nitida a redução de desempenho com 1 e 4 threads, o tempo foi somente aumentando. Isso ocorre porque o GIL força as threads a alternarem a execução sequencialmente, por isso ao adicionar uma segunda thread apenas dividiu o mesmo tempo de processador, gerando uma falsa percepção de paralelismo. O cenário crítico ocorreu com 2 threads, onde a média saltou para 0,390 s, um aumento de quase 10%. Esse pico de lentidão é explicado pela intensa disputa pelo bloqueio do GIL, quatro threads tentando adquirir e liberar o mesmo *lock* simultaneamente geram um *overhead* massivo de troca de contexto no sistema operacional, fazendo o processador gastar mais tempo gerenciando a concorrência do que computando os números primos.

5.2 Implicações no Contexto de Sistemas Operacionais

Os resultados observados mostram os conceitos teóricos de sistemas operacionais, evidenciando as diferenças arquiteturais entre os modelos de concorrência. Threads compartilham o mesmo espaço de endereçamento e são unidades mais leves, o que explica por que o cenário de 1 thread (0,388 s) iniciou de forma mais rápida do que 1 processo (0,423 s), poupando o sistema de chamadas de sistema dispendiosas e da duplicação de estruturas de memória.

Entretanto, quando é exigido poder computacional bruto, o isolamento de memória dos processos torna-se uma vantagem crucial. A distribuição de processos pelo escalonador do Sistema Operacional permite que múltiplos núcleos de CPU operem em frequências máximas de forma verdadeiramente paralela, sem colisões de espaço de usuário.

O desempenho perdido pelo uso de threads com 2 workers atua como um excelente laboratório empírico sobre o custo da troca de contexto sobre o TCB (Thread Control Block). Embora o hardware possua capacidade de processar essas threads em núcleos distintos, a restrição causada pelo GIL acaba mudando o comportamento padrão esperado do escalonador do SO, transformando threads de alta afinidade em um gargalo de concorrências.

5.3 Cenários de Aplicação Prática

A partir dos resultados obtidos e da discussão teórica, é possível alcançar recomendações práticas sobre qual escolha fazer entre processos e threads em Python:

- Tarefas CPU-intensivas (processamento de dados, criptografia, cálculos matemáticos): Dê preferência a *multiprocessing* (processos). Como demonstrado pelos dados, o ganho de tempo chega a superar 42% de eficiência em relação às threads quando escalado para 4 workers, pois é a única abordagem que quebra a barreira do GIL e faz uso real de processadores multi-core.
- Tarefas I/O-intensivas (requisições de rede, leitura/escrita de arquivos, chamadas de banco de dados): O uso de *threading* (threads) é adequado nestes cenários, já que o GIL é liberado durante as esperas de operações de entrada e saída, permitindo eficiência sem o custo de memória de um processo.
- Escalabilidade Concorrente: É melhor evitar o aumento sem controle de threads no Python para tarefas computacionais, visto que o aumento de workers amplifica a disputa pelo GIL, diminuindo o desempenho para níveis piores do que uma execução.

6 CONCLUSÃO

Este trabalho investigou experimentalmente as diferenças de desempenho entre processos e threads em Python, utilizando o cálculo de números primos como carga de trabalho *CPU-intensiva* em ambiente Windows. Os experimentos realizados com 1, 2 e 4 workers, estruturados em dez repetições para cada cenário, produziram resultados sólidos que alinham a prática experimental às restrições arquiteturais da linguagem CPython.

O principal achado do trabalho foi o impacto do *Global Interpreter Lock* (GIL) no paralelismo real por subprocessos. Enquanto a abordagem por processos entregou uma redução de tempo escalar, provando que múltiplos interpretadores Python independentes conseguem usufruir da arquitetura multi-core do processador, a abordagem por threads mostrou ser ineficiente. O aumento no número de threads não apenas falhou em reduzir o tempo de execução como, no cenário de 2 workers, causou uma piora de desempenho de aproximadamente 08% devido ao *overhead* de disputa pelo *lock* do interpretador.

Conclui-se que para sistemas que demandam processamento computacional massivo em Python, apesar de exigir maior consumo de memória inicial, os processos são a escolha correta e de maior desempenho.

Como trabalhos futuros, sugere-se a replicação deste experimento expandindo o volume de dados computados (limites maiores de números primos ou outros

experimentos ainda mais intensos) para avaliar o limite dos processos. Sugere-se também a investigação comparativa em sistemas baseados em Unix (como Linux), onde o modelo de criação de processos (*fork*) elimina parte do *overhead* de inicialização detectado no ecossistema Windows (*spawn*). Por fim, é recomendada a análise desses mesmos algoritmos sob a perspectiva da nova diretriz PEP 703, que visa tornar o GIL opcional em versões futuras do Python, o que poderá alterar drasticamente o padrão observado neste estudo.

REFERÊNCIAS

BEAZLEY, David. Understanding the Python GIL. PyCon 2010, Atlanta, 2010. Disponível em: <https://www.dabeaz.com/python/UnderstandingGIL.pdf>. Acesso em: maio 2025.

PYTHON SOFTWARE FOUNDATION. multiprocessing — Process-based parallelism. Python 3 Documentation. Disponível em: <https://docs.python.org/3/library/multiprocessing.html>. Acesso em: maio 2025.

PYTHON SOFTWARE FOUNDATION. threading — Thread-based parallelism. Python 3 Documentation. Disponível em: <https://docs.python.org/3/library/threading.html>. Acesso em: maio 2025.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. Operating System Concepts. 10. ed. Hoboken: Wiley, 2018.

STALLINGS, William. Operating Systems: Internals and Design Principles. 9. ed. Hoboken: Pearson, 2018.

TANENBAUM, Andrew S.; BOS, Herbert. Modern Operating Systems. 4. ed. Hoboken: Pearson, 2015.